

Introduction to Vector Search 2.0...

Language ▾



1 Introduction (#0)

2 Building TheLook Fashion Search (#1)

3 Congratulations (#2)

What is Vector Search 2.0?

[Vector Search 2.0](https://cloud.google.com/vertex-ai/docs/vector-search-2/overview) (<https://cloud.google.com/vertex-ai/docs/vector-search-2/overview>) is Google Cloud's fully managed, self-tuning vector database built on Google's [ScaNN \(Scalable Nearest Neighbors\)](https://github.com/google-research/google-research/tree/master/scann) (<https://github.com/google-research/google-research/tree/master/scann>) algorithm - the same technology powering Google Search, YouTube, and Google Play.

Key Differentiators

- **Zero Indexing to Billion-Scale Index:** Start developing immediately with zero indexing time using [kNN \(k-Nearest Neighbors\)](https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm) (https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm), then scale to billions of vectors with millisecond latency with Google-scale [ANN \(Approximate Nearest Neighbor\)](https://en.wikipedia.org/wiki/Nearest_neighbor_search#Approximate_nearest_neighbor) (https://en.wikipedia.org/wiki/Nearest_neighbor_search#Approximate_nearest_neighbor) indexes for production - all with the same API and same dataset
- **Unified Data Storage:** Store both [vector embeddings](https://cloud.google.com/vertex-ai/docs/generative-ai/embeddings) (<https://cloud.google.com/vertex-ai/docs/generative-ai/embeddings>) and user provided data together (no separate database or feature store needed)
- **Auto-Embeddings:** Automatically generate semantic embeddings using [Vertex AI embedding models](https://docs.cloud.google.com/vertex-ai/generative-ai/docs/embeddings/get-text-embeddings#google-models) (<https://docs.cloud.google.com/vertex-ai/generative-ai/docs/embeddings/get-text-embeddings#google-models>)
- **Built-in Full Text Search:** Provides a built-in [full-text search](https://cloud.google.com/discover/what-is-full-text-search?e=48754805) (<https://cloud.google.com/discover/what-is-full-text-search?e=48754805>) without needing to generate sparse embeddings by yourself. You can also choose to use your own sparse embeddings (e.g., BM25, SPLADE) with Vector Search for a customized full-text search.
- **Hybrid Search:** Combine semantic and keyword/token-based search in a single query with intelligent [RRF](https://plg.uwaterloo.ca/~gvcormac/cormacksigir09-rrf.pdf) (<https://plg.uwaterloo.ca/~gvcormac/cormacksigir09-rrf.pdf>) ranking
- **Self-Tuning:** Auto-optimized performance without manual configuration
- **Enterprise-Ready:** Built-in scalability, security, and compliance

Core Architecture

Vector Search 2.0 has three main components:

1. [Collections](https://cloud.google.com/vertex-ai/docs/vector-search-2/collections/collections)

(<https://cloud.google.com/vertex-ai/docs/vector-search-2/collections/collections>): Schema-enforced containers for your data

2. [Data Objects](https://cloud.google.com/vertex-ai/docs/vector-search-2/data-objects/data-objects)

(<https://cloud.google.com/vertex-ai/docs/vector-search-2/data-objects/data-objects>): Individual items with data and vector embeddings

3. [Indexes](https://cloud.google.com/vertex-ai/docs/vector-search-2/indexes/indexes) (<https://cloud.google.com/vertex-ai/docs/vector-search-2/indexes/indexes>): Instant nearest neighbor search your data with kNN. For low latency nearest neighbor search use an ANN index.

- **Start fast:** Use kNN immediately with zero setup time - perfect for development and small datasets
- **Scale to production:** Use ANN indexes for billion-scale search with sub-second latency powered by ScaNN algorithm

Let's explore each concept with hands-on examples!

[Report a mistake \(#\)](#)[Next \(#\)](#)